

Структуры на максималках: ref struct, readonly ref, in-параметры

C# in depth



Проверить, идет ли запись

Меня хорошо видно & слышно?



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта модуля



СТАРТ

Организация памяти в
.NET

Сборщик мусора (GC)

Низкоуровневая работа
с памятью

Продвинутое
управление памятью

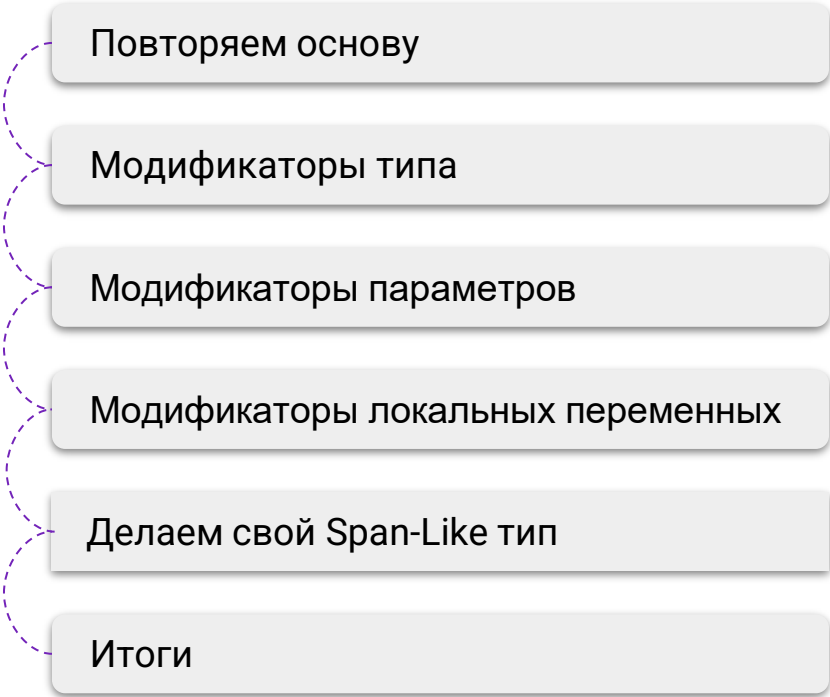
Структуры на
максималках

Работа с
неуправляемыми
ресурсами

ФИНИШ



Маршрут вебинара



Повторяем основу

Модификаторы типа

Модификаторы параметров

Модификаторы локальных переменных

Делаем свой Span-Like тип

Итоги



Цели вебинара

К концу занятия вы сможете

1. Минимизировать копирование структур.
2. Использовать `ref struct` для высокопроизводительной работы на стеке.
3. Создать API, сочетающее безопасность и скорость.



Повторяем основы

Struct:

- Не явно наследуется от `System.ValueType`;
- Не может наследовать или быть унаследованной;
- Может размещаться на стеке
- Может быть запакована и размещена в куче
- Может быть часть ссылочного типа в куче
- Для получения значения запакованной структуры требуется распаковка;
- Как часть ссылочного типа может находиться на куче без упаковки;
- Может реализовывать интерфейсы;
- Хранит не ссылки, а значения;
- Передача и возвращение из метода происходит через копирование*;

До C# версии 10 и 11 действовали строгие правила инициализации структур.

* Не учитываем модификаторы

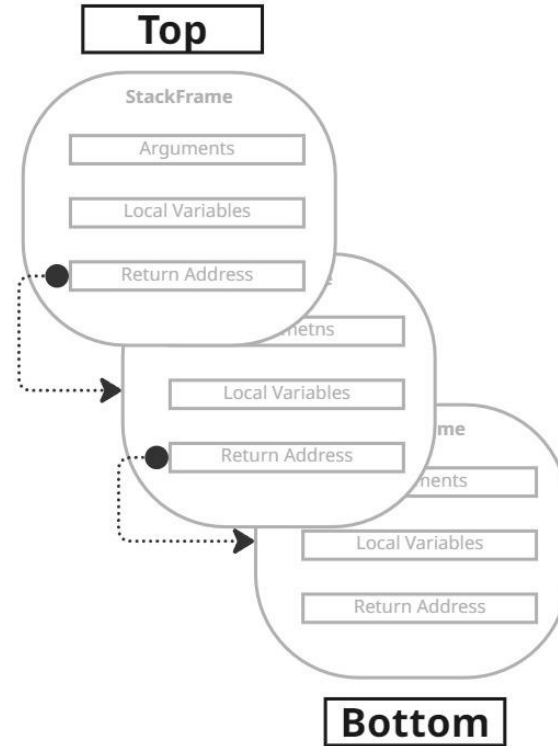
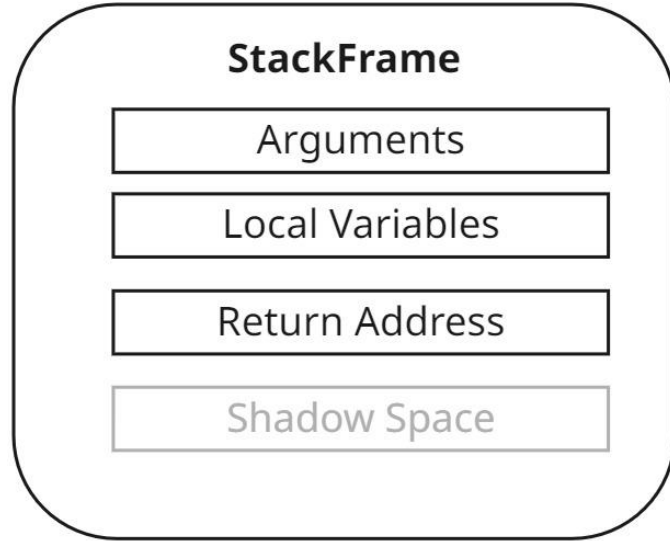


Call stack:

- Создаётся для каждого потока (thread) в CLR;
- Управляется CLR (CLR в части операций использует API OS, прослойка);
- Подчиняется контракту ABI — соглашению о вызовах, которое определяет передачу аргументов, сохранение регистров, выравнивание и так далее;
- Имеет ограниченный размер: стек каждого потока ограничен (по умолчанию, либо конфигурацией);
- Состоит из последовательности StackFrame (активационных записей): каждый кадр соответствует не завершённому ещё вызову метода;
- В вершине (top) стека находится стек-фрейм метода, который в данный момент выполняется (последний вызов);
- В основании (bottom) стека находится кадр первого метода управления, обычно точка входа managed-приложения; в консольном приложении это может быть метод Main, но под ним могут быть ещё кадры runtime/загрузки CLR.



Call Stack

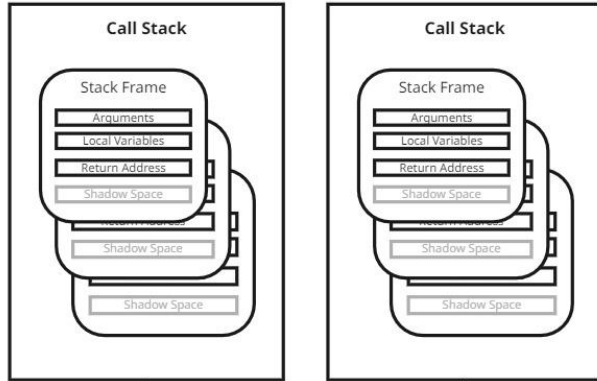


OS

Process: DotnetApp1

Thread ID: 13

Thread ID: 2

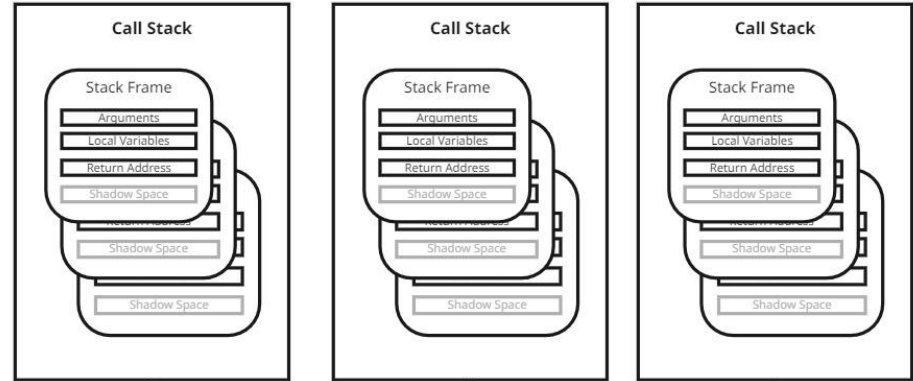


Process: DotnetApp2

Thread ID: 7

Thread ID: 22

Thread ID: 22



Модификаторы типа

Рассмотрим три варианта:

- ref
- readonly
- ref readonly

ref struct:

Начиная с NetFramework 4.7.2 особый вид value type, который ограничен размещением на стеке. Он был введён для высокопроизводительных сценариев, например работы с `Span<T>` и временными буферами, чтобы избежать аллокаций в managed heap. Такой тип даёт более строгие гарантии времени жизни и безопасности памяти по сравнению с обычными struct, что делает его важным инструментом при создании эффективных, низкоуровневых API в .NET.



Ограничения ref struct:

Не может быть
элементом массива

Не может быть **полем**
non-ref struct типа

Запрещена **упаковка**
(boxing)

Лямбды или
локальные функции

Async и **yield return**
(до C#13 *)

Реализация
интерфейсов
(до C#13*)

Аргумент generic type
(до C#13)

Используются до await
И yield return

Реализация по
умолчанию и каст
недоступны

readonly struct:

В C# модификатор `readonly` для структуры (`readonly struct`) гарантирует, что структура неизменяема после создания: все её поля должны быть `readonly` или свойства должны быть только с `get` (или `init`-сеттером). Это позволяет компилятору применять оптимизации, например избегать ненужных копирований (`defensive copies`), и повышает безопасность и предсказуемость кода в высокопроизводительных сценариях.



Ограничения readonly struct:

Все поля readonly

Свойства только **get**
или **init**

После инициализации
в конструкторе поля
неизменны

Readonly struct не делает все
вложенные объекты
неизменяемыми

readonly ref struct:

readonly ref struct — это тип-значение, объединяющий два свойства: он жестко привязан к стеку (как ref struct) и неизменяемый (как readonly struct), то есть экземпляр такого типа не может быть перемещён в кучу и все его поля после инициализации не меняются. Примером такого типа служит ReadOnlySpan<T>, который хранит ссылку на участок памяти + длину, работает без копирования в hot path, и при этом гарантирует, что состояние не изменится после создания.



Модификаторы параметров

Рассмотрим четыре варианта:

- ref
- ref readonly
- in
- out

ref:

- Аргумент должен быть инициализирован перед вызовом;
- Метод может изменять аргумент;
- Метод не обязан присваивать значение;
- Аргумент передаётся по ссылке, копирование не происходит;

В отличие от `in` и `ref readonly` не создаёт копию т.к. всегда указывает на самостоятельную переменную.

Модификатор `ref` указывается всегда и для параметров и для аргументов.

Не принимает в качестве аргумента `readonly` типы или поля.



ref readonly:

- Аргумент должен быть инициализирован перед вызовом;
- Метод не может изменять аргумент;
- Метод не обязан присваивать значение;
- Аргумент передаётся по ссылке, копирование не происходит*;

Фактически близнец `in`. Для CLR разницы нету, отличие только в проверках компилятора.

Допускает для аргумента как `ref` так и `in` модификатор.

*На самом деле копирование может происходить в нескольких случаях:

- Если структура не `readonly struct` и есть внутренний метод изменяющий её состояние;
- При необходимости приведения типа;
- Метод не объявлен как `readonly`;

```
using System;

public class C {
    public void M()
    {
        M1(new W());
    }

    public int M1(ref readonly W x)
    {
        x.Mutate();
        return x.F * x.F;
    }
}

public struct W
{
    public int F {get;set;}
    public int Mutate() => 2;
}
```

```
[assembly: SecurityPermission(SecurityAction.RequestMinimum)]
[assembly: AssemblyVersion("0.0.0.0")]
[module: UnverifiableCode]
[module: RefSafetyRules(11)]

public class C
{
    public void M()
    {
        W x = default(W);
        M1(ref x);
    }

    public int M1([In][RequiresLocation] ref W x)
    {
        W w = x;
        w.Mutate();
        return x.F * x.F;
    }
}
```

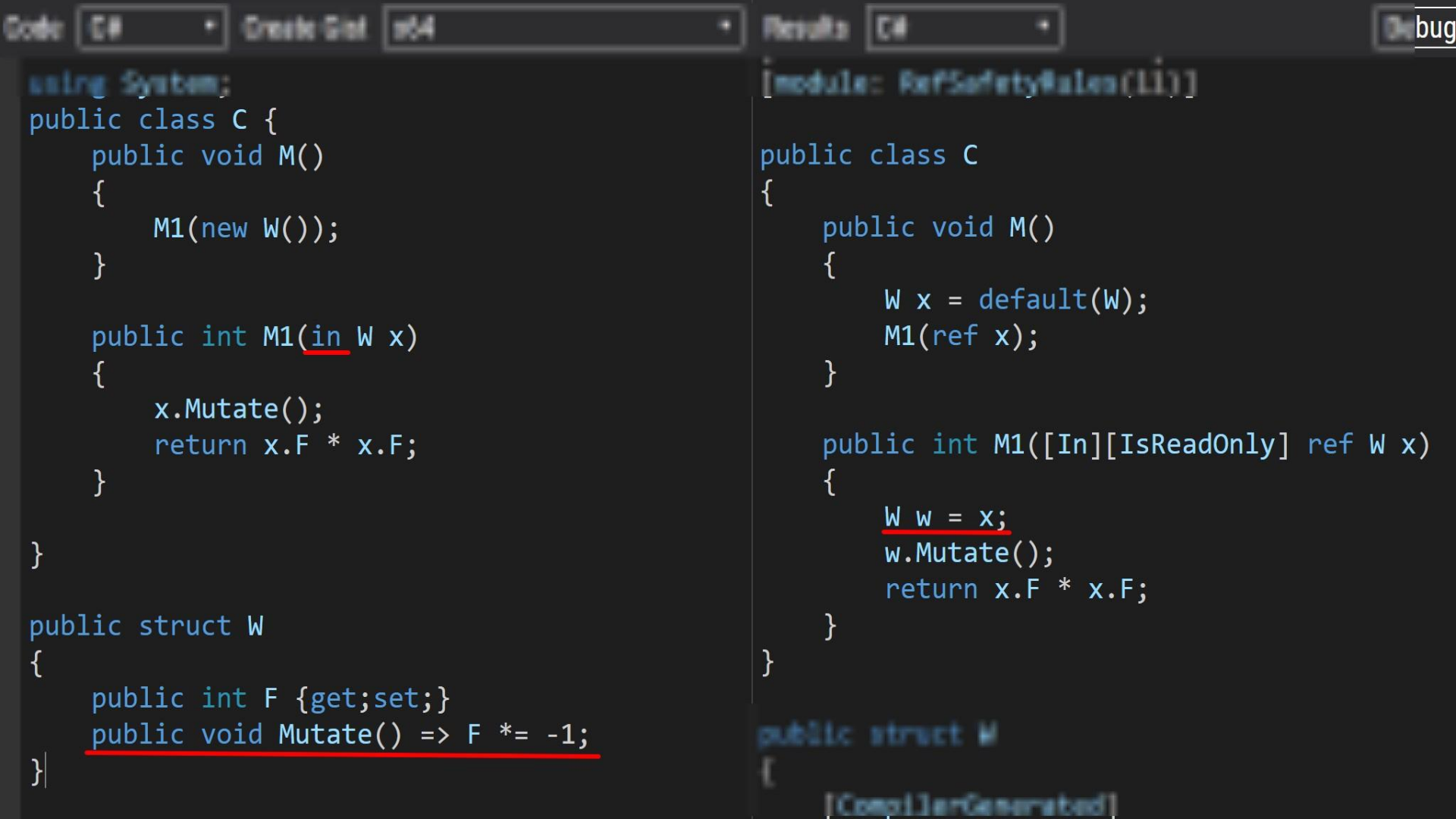
in:

- Аргумент должен быть инициализирован перед вызовом;
- Метод не может изменять аргумент;
- Метод не обязан присваивать значение;
- Аргумент передаётся по ссылке, копирование не происходит*;

*На самом деле копирование может происходить в нескольких случаях:

- Если структура не `readonly struct` и есть внутренний метод изменяющий её состояние;
- При необходимости приведения типа;
- Метод не объявлен как `readonly`;





```
using System;
public class C {
    public void M()
    {
        M1(new W());
    }

    public int M1(in W x)
    {
        x.Mutate();
        return x.F * x.F;
    }
}

public struct W
{
    public int F {get;set;}
    public readonly int Mutate() => 2;
}
```

[module: RefSafetyRules(11)]

```
public class C
{
    public void M()
    {
        W x = default(W);
        M1(ref x);
    }

    public int M1([In][IsReadOnly] ref W x)
    {
        x.Mutate();
        return x.F * x.F;
    }
}
```

public struct W

```
{
    [CompilerGenerated]
    [DebuggerBrowsable(DebuggerBrowsableState.Never)]
    private int <F>__BackingField;
```

out:

- Аргумент должен быть инициализирован перед вызовом;
- Метод может изменять аргумент;
- Метод обязан присвоить значение;
- Аргумент передаётся по ссылке, копирование не происходит;

Модификаторы локальных переменных

Рассмотрим два варианта:

- ref
- ref readonly

ref:

Локальная переменная с модификатором `ref` создаёт `alias` (ссылку) на существующую структуру (или поле/элемент массива), то есть не копирует её. Когда мы объявляем «`ref SomeStruct s = ref other;`», все чтения и записи через `s` воздействуют на оригинал (`other`). Это полезно, когда нужно избежать копирования больших структур и/или модифицировать данные напрямую. Ограничения: аргумент, из которого берётся `ref`, должен быть адресуемым (переменная, поле, элемент массива или `ref return`). Нельзя объявить `ref`-локальную на выражение, литерал или свойство без `ref return`. Также `ref struct` нельзя сохранять в куче и нельзя использовать в `async/yield` контекстах.



ref readonly:

Локальная переменная с модификатором `ref readonly` — это тоже `alias` на существующую структуру, но только для чтения. При объявлении «`ref readonly SomeStruct r = ref other;`» мы получаем ссылку, через которую нельзя менять поля структуры. Это даёт защиту: мы знаем, что через этот `alias` оригинал не будет изменён.

Замечания: те же адресуемые источники применимы (переменная/поле/элемент массива/`ref return`). Если структура не `readonly`, то вызов нестатических методов, не помеченных `readonly`, может привести к защитной копии (`defensive copy`) этого `ref readonly`-локала. И как с `ref`, так и с `ref readonly`, `ref struct` типы имеют дополнительное ограничение: нельзя использовать `ref readonly` локальную переменную с `ref struct` в `async/iterator` методах или сохранять её в куче.



Делаем свой Span-Like тип

MySpan поля, свойства, индексатор:

```
public ref struct MySpan<T> ..  
    // Byref на первый элемент (управляемая ссылка)  
    private readonly ref T _reference;  
    private readonly int _length;  
    public int Length => _length;  
    public bool IsEmpty => _length == 0;  
  
    // Индексатор с проверкой границ  
    public ref T this[int index]  
    {  
        [MethodImpl(MethodImplOptions.AggressiveInlining)]  
        get  
        {  
            if ((uint)index >= (uint)_length) throw new IndexOutOfRangeException();  
            return ref Unsafe.Add(ref _reference, index);  
        }  
    }  
}
```

MySpan конструкторы

```
// Конструктор для создания ref ссылки из массива
public MySpan(T[] array)
{
    if (array is null) throw new ArgumentNullException(nameof(array));
    _reference = ref (array.Length > 0 ? ref array[0] : ref Unsafe.NullRef<T>());
    _length = array.Length;
}

// Приватный конструктор из произвольного byref (для Slice)
private MySpan(ref T startRef, int length)
{
    _reference = ref (length > 0 ? ref startRef : ref Unsafe.NullRef<T>());
    _length = length;
}
```



MySpan метод Clear:

```
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void Clear()
{
    if (_length <= 0) return;

    if (!RuntimeHelpers.IsReferenceOrContainsReferences<T>())
    {
        ref byte firstByte = ref Unsafe.As<T, byte>(ref _reference);
        Unsafe.InitBlockUnaligned(ref firstByte, 0, (uint)(_length *
        Unsafe.SizeOf<T>()));
    }
    else
    {
        for (int i = 0; i < _length; i++)
            Unsafe.Add(ref _reference, i) = default!;
    }
}
```

MySpan метод Fill:

```
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public void Fill(T value)
{
    if (_length <= 0) return;

    if (!RuntimeHelpers.IsReferenceOrContainsReferences<T>())
    {
        // 1) Присвоение value в первый элемент
        this[0] = value;

        // 2) Удвоениями копируем уже заполненный префикс
        int filled = 1;
        int size = Unsafe.SizeOf<T>();
        ref byte baseRef = ref Unsafe.As<T, byte>(ref _reference);

        while (filled < _length)
        {
            int copy = Math.Min(filled, _length - filled);
            ref byte dst = ref Unsafe.Add(ref baseRef, filled * size);
            Unsafe.CopyBlockUnaligned(ref dst, ref baseRef, (uint)(copy * size));
            filled += copy;
        }
    }
    else
    {
        for (int i = 0; i < _length; i++)
            Unsafe.Add(ref _reference, i) = value!;
    }
}
```

MySpan метод Slice:

```
[MethodImpl(MethodImplOptions.AggressiveInlining)]
public MySpan<T> Slice(int start, int length)
{
    if ((uint)start > (uint)_length || (uint)length > (uint)(_length - start))
        throw new ArgumentOutOfRangeException();

    ref T startRef = ref Unsafe.Add(ref _reference, start);
    return new MySpan<T>(ref startRef, length);
}
```



Span<int> метод Clear:

```
public int Span_Clear()
{
    var span = _buffer.AsSpan(); // локальный Span
    span.Clear();
    return span.Length;
}
```



Span<int> метод Fill:

```
public int Span_Fill()
{
    var span = _buffer.AsSpan();
    span.Fill(FillValue);
    return span[0];
}
```

Span<int> метод Slice:

```
public int Span_Slice_Create()
{
    var span = _buffer.AsSpan();
    int sum = 0;
    int step = Math.Max(1, N / Slices);
    for (int s = 0; s < Slices; s++)
    {
        int start = (s * step) % Math.Max(1, N);
        int len = Math.Min(step, N - start);
        var sub = span.Slice(start, len);
        sum += sub.Length;
    }
    return sum;
}
```


Int[] метод Clear:

```
public int Array_Clear()  
{  
    Array.Clear(_buffer, 0, _buffer.Length);  
    return _buffer.Length;  
}
```



Int[] метод Fill:

```
public int Array_Fill()  
{  
    Array.Fill(_buffer, FillValue);  
    return _buffer[0];  
}
```



Int[] метод Slice:

```
public int Array_Slice_Create()
{
    int sum = 0;
    int step = Math.Max(1, N / Slices);
    for (int s = 0; s < Slices; s++)
    {
        int start = (s * step) % Math.Max(1, N);
        int len = Math.Min(step, N - start);
        var seg = new ArraySegment<int>(_buffer, start, len);
        sum += seg.Count;
    }
    return sum;
}
```



Бенчмарк:

Сравниваем int[], Span<int> и MySpan<int>

```
BenchmarkDotNet v0.15.2, Windows 10 (10.0.19045.4957/22H2/2022Update)
Intel Core i7-10700 CPU 2.90GHz, 1 CPU, 16 logical and 8 physical cores
.NET SDK 9.0.305
[Host] : .NET 9.0.9 (9.0.925.41916), X64 RyuJIT AVX2
.NET 9.0 : .NET 9.0.9 (9.0.925.41916), X64 RyuJIT AVX2
```

```
Job=.NET 9.0 Runtime=.NET 9.0
```

Method	N	Slices	FillValue	Mean	Error	StdDev	Allocated
Array_Clear	1000000	256	-7	64,669.5 ns	167.43 ns	139.81 ns	-
Array_Fill	1000000	256	-7	94,727.5 ns	301.37 ns	281.90 ns	-
Array_Slice_Create	1000000	256	-7	561.0 ns	1.29 ns	1.14 ns	-
Span_Clear	1000000	256	-7	64,587.4 ns	169.53 ns	158.58 ns	-
Span_Fill	1000000	256	-7	94,719.1 ns	278.22 ns	232.32 ns	-
Span_Slice_Create	1000000	256	-7	615.6 ns	1.15 ns	1.07 ns	-
MySpan_Clear	1000000	256	-7	64,566.0 ns	173.41 ns	162.20 ns	-
MySpan_Fill	1000000	256	-7	110,251.4 ns	417.09 ns	369.74 ns	-
MySpan_Slice_Create	1000000	256	-7	508.3 ns	1.34 ns	1.25 ns	-
Array_Clear	1000000	256	123	64,440.8 ns	174.50 ns	154.69 ns	-
Array_Fill	1000000	256	123	94,687.8 ns	303.22 ns	283.63 ns	-
Array_Slice_Create	1000000	256	123	562.1 ns	0.93 ns	0.78 ns	-
Span_Clear	1000000	256	123	65,171.9 ns	621.43 ns	581.29 ns	-
Span_Fill	1000000	256	123	95,217.3 ns	226.80 ns	177.07 ns	-
Span_Slice_Create	1000000	256	123	619.5 ns	0.17 ns	0.14 ns	-
MySpan_Clear	1000000	256	123	64,721.7 ns	68.39 ns	57.11 ns	-
MySpan_Fill	1000000	256	123	110,692.0 ns	367.71 ns	343.96 ns	-
MySpan_Slice_Create	1000000	256	123	509.0 ns	0.75 ns	0.67 ns	-

Подводим итоги
